

Embedded Systems LAB 3 – FPGAs as Accelerators

Ioannis Papadopoulos, Maria Ilektra Tzormpatzaki
Student IDs: 03899, 03993

Date: 1/6/2026
Board: Xilinx Zynq-7000 FPGA (Zedboard)

Contents

1	LSAL Implementation	2
1.1	Initial LSAL Implementation on the x86 CPU	2
1.2	Profile and Optimization of the x86 CPU Implementation: Roofline Analysis	4
2	Arm CPU	9
2.1	LSAL implementation on the Arm CPU	9
2.2	Profiling and Performance Analysis on the Arm CPU	9
3	Hardware	10
3.1	Step 1: Basic Hardware Porting (<code>lsal_serial</code>)	10
3.2	Step 2: Anti-Diagonal Scheduling and Datapath Unrolling (<code>lsal_diag</code>)	12
3.3	Step 3: Advanced Datapath, Reduction Tree, and Memory Bandwidth Optimization	15
3.4	Step 4: Memory Bandwidth Minimization and Output Compression	17
3.5	Step 5: Maximizing AXI Bus Potential with Ultra-Wide Input Fetches	19

1 LSAL Implementation

1.1 Initial LSAL Implementation on the x86 CPU

The implementation of the LSAL algorithm on the x86 CPU evolved through several stages. The initial version, `lsal.cpp`, established the core logic of the Smith–Waterman local alignment. The `compute_matrices` function was designed with a nested loop structure where the outer loop iterates over the database of length M and the inner loop over the query of length N .

The core of the function lies in determining the similarity score for each cell (i, j) based on its neighbours. The algorithm evaluates three possible moves: North-West, North, and West. It selects the one that yields the highest score, while ensuring that the value never drops below zero.

Diagonal Move (Match/Mismatch)

The process begins by checking the North-West neighbour. It compares the character from the query string with the character from the database string. If they match, a `MATCH` score is added to the diagonal value, otherwise a `MISS_MATCH` penalty is applied. If this result is greater than the current value, initially zero, the cell is updated, the value is stored temporarily in `val`, and the direction is set to `NORTH_WEST`.

Vertical Move (Insertion/Gap in Query)

Next, the algorithm evaluates the North neighbour. It adds a gap penalty, `GAP_i`, to the value of the cell directly above. If this new score, `test_n`, is higher than the current `val`, it overwrites it, and the direction is updated to `NORTH`.

Horizontal Move (Deletion/Gap in Database)

Finally, the West neighbour is checked. A gap penalty, `GAP_d`, is added to the value of the cell to the left. If this result, `test_w`, is the highest among all options, the cell receives this value, and the direction is set to `WEST`.

Store the Results in the Matrices and Track the Maximum Value

The algorithm stores the current highest value, `val`, in the `similarity_matrix` and each corresponding direction in the `direction_matrix`. If the current `val` is the highest value that has occurred so far, `max_val` and `max_index` are updated. This way, `max_index` represents the position of the highest value in the similarity matrix.

```
// Check NORTH_WEST
match_score = (string1[i-1] == string2[j-1]) ? MATCH : MISS_MATCH;
test_nw = northwest + match_score;
if (test_nw > val) {
    val = test_nw;
    dir = NORTH_WEST;
}

// Check NORTH
test_n = north + GAP_i;
if (test_n > val) {
```

```

    val = test_n;
    dir = NORTH;
}

// Check WEST (Gap in Database)
test_w = west + GAP_d;
if (test_w > val) {
    val = test_w;
    dir = WEST;
}

// Store the results in the matrices
similarity_matrix[index] = val;
direction_matrix[index] = dir;

// Track the maximum value
if (val > max_val) {
    max_val = val;
    *max_index = index;
}

```

The difference between `lsal.cpp` and the other version tested, `lsal_plus1.cpp`, is the explicit boundary checks within the inner loop:

```

north = (j > 0) ? similarity_matrix[index - N] : 0;
west = (i > 0) ? similarity_matrix[index - 1] : 0;
northwest = (i > 0 && j > 0) ? similarity_matrix[index - N - 1] : 0;

```

These conditional checks were necessary to handle the first row and column, but introduced significant branching overhead inside the most frequently executed part of the code. Padding was tested for boundary removal in order to eliminate the overhead of conditional branching. By allocating the similarity and direction matrices with a size of $(N + 1)(M + 1)$, the first row and column were initialized to zero. This allowed the inner loop to start from index 1, enabling direct memory access without the need for `if` statements:

```

north = similarity_matrix[index - N - 1];
west = similarity_matrix[index - 1];
northwest = similarity_matrix[index - N - 2];

```

Further refinement involved loop invariant code motion, implemented in `lsal_outside.cpp`. This optimization moved calculations that do not change across iterations of i outside the inner loop. In this version, the row offset, $j * N$, is pre-calculated and the current character of the database string, `string2[j]`, is stored in a temporary variable. This reduced redundant multiplications and array indexing operations within the inner loop.

```

// Outer loop iterates over the Database (Rows)
for (j = 0; j < M; j++) {
    string2_temp = string2[j];
    row_offset = j * N;

    // Inner loop iterates over the Query (Columns)

```

```

for (i = 0; i < N; i++) {
    index = row_offset + i;
    ...
    match_score = (string1[i] == string2_temp) ? MATCH : MISS_MATCH;
}
}

```

1.2 Profile and Optimization of the x86 CPU Implementation: Roofline Analysis

In this section, we first compare the performance of `lsal.cpp` and `lsal_plus1.cpp` using execution times and Intel Advisor data. The objective is to determine which implementation serves as a more efficient baseline. Once the superior version is identified, it is compared against the `lsal_outside.cpp` variant to evaluate the impact of moving invariant calculations outside the inner loop. This step-by-step comparison allows us to isolate the benefits of each optimization before reaching the final implementation.

Table 1: Execution-time comparison between `lsal.cpp` and `lsal_plus1.cpp`.

Inputs	Measurement	<code>lsal.cpp</code>	<code>lsal_plus1.cpp</code>
$N = 32, M = 65536$	1	0.016622	0.015866
	2	0.016609	0.015900
	3	0.016827	0.015695
	4	0.016818	0.015730
	5	0.016514	0.015842
	6	0.016725	0.016156
	7	0.016552	0.015708
	8	0.016828	0.015812
	9	0.016822	0.015847
	10	0.016614	0.015879
	Average	0.016693	0.015844
$N = 256, M = 65536$	1	0.130924	0.127901
	2	0.130886	0.104250
	3	0.130900	0.128004
	4	0.130634	0.128107
	5	0.130222	0.127840
	6	0.130663	0.103087
	7	0.130254	0.128296
	8	0.130484	0.127821
	9	0.131031	0.127668
	10	0.130411	0.103881
	Average	0.130641	0.120686
$N = 256, M = 300000$	1	0.598908	0.583025
	2	0.596296	0.582993
	3	0.595708	0.583961
	4	0.596933	0.583500
	5	0.463800	0.456042
	6	0.597326	0.455213
	7	0.596759	0.583296
	8	0.595789	0.583438
	9	0.465943	0.583392
	10	0.596456	0.583799
	Average	0.570392	0.557866

Based on our benchmarks, `lsal_plus1.cpp` consistently outperforms the baseline `lsal.cpp` across all tested input sizes. This performance gain is also clearly visualized in the Intel Advisor roofline model, where the data points for `lsal_plus1` are positioned further upward and to the right compared to the `lsal` implementation.

The `lsal_plus1` implementation is depicted with a square, and `lsal` with a circle.

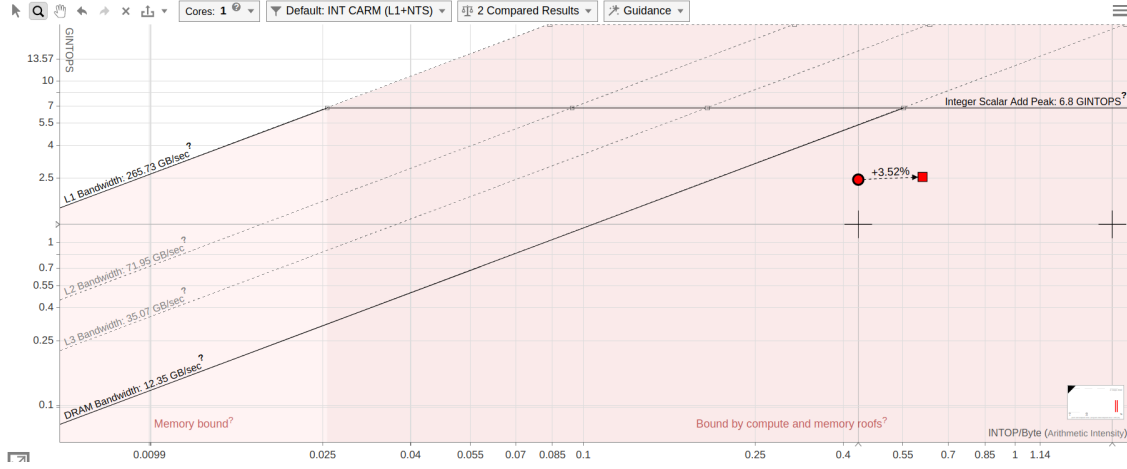


Figure 1: `lsal` and `lsal_plus1` with $N = 32$ and $M = 65536$.

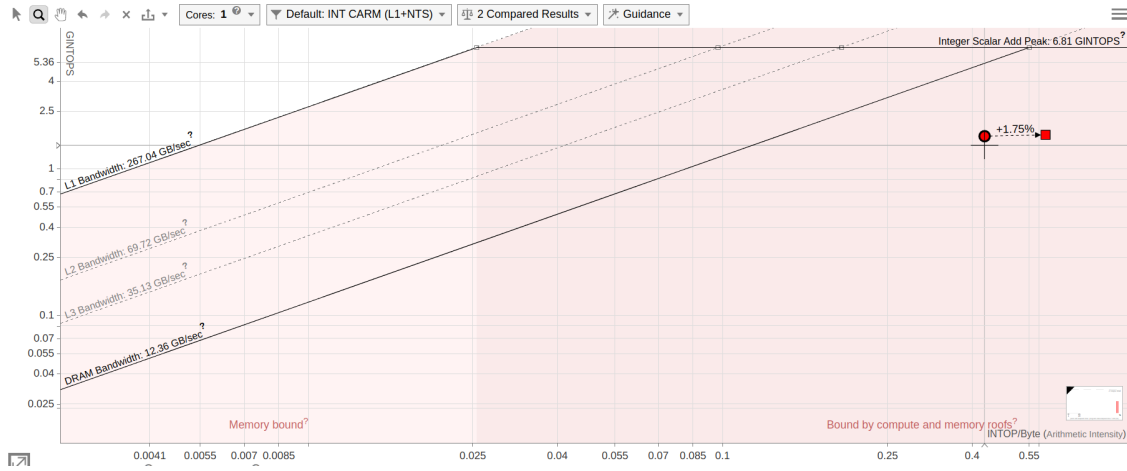


Figure 2: `lsal` and `lsal_plus1` with $N = 256$ and $M = 65536$.

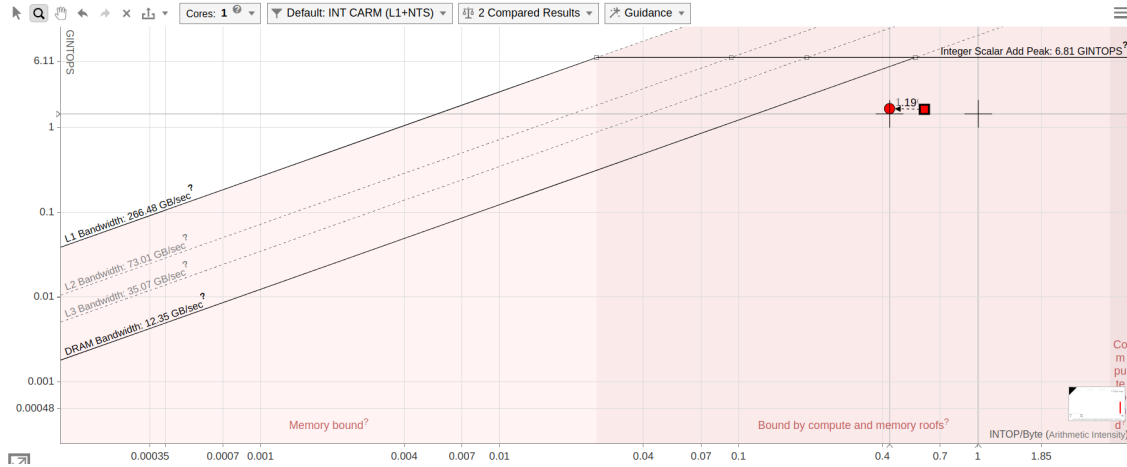


Figure 3: `lsal` and `lsal_plus1` with $N = 256$ and $M = 300000$.

The upward shift indicates higher throughput in GINTOPS/sec, proving that the removal of conditional boundary checks reduced the execution time per operation. Simultaneously, the rightward shift signifies an increase in arithmetic intensity (AI). By utilizing matrix padding, the algorithm minimizes the overhead of memory-bound logic and branching, allowing the CPU to process data more efficiently. In conclusion, Intel Advisor confirms that `lsal_plus1` is the superior foundation.

Following the same evaluation pattern, the `lsal_outside` implementation, which incorporates matrix padding, comes out as the most efficient version.

Table 2: Execution-time comparison between `lsal_plus1.cpp` and `lsal_outside.cpp`.

Inputs	Measurement	<code>lsal_plus1.cpp</code>	<code>lsal_outside.cpp</code>
$N = 32, M = 65536$	1	0.015866	0.014802
	2	0.015900	0.014248
	3	0.015695	0.014815
	4	0.015730	0.014358
	5	0.015842	0.014452
	6	0.016156	0.014259
	7	0.015708	0.014265
	8	0.015812	0.014862
	9	0.015847	0.014235
	10	0.015879	0.015059
	Average	0.015844	0.014536
$N = 256, M = 65536$	1	0.127901	0.117696
	2	0.104250	0.117410
	3	0.128004	0.117685
	4	0.128107	0.117671
	5	0.127840	0.117654
	6	0.103087	0.117640
	7	0.128296	0.117553
	8	0.127821	0.1176553
	9	0.127668	0.117685
	10	0.103881	0.117640
	Average	0.120686	0.117629
$N = 256, M = 300000$	1	0.583025	0.539591
	2	0.582993	0.538613
	3	0.583961	0.538483
	4	0.583500	0.538534
	5	0.456042	0.538022
	6	0.455213	0.538500
	7	0.583296	0.538854
	8	0.583438	0.538096
	9	0.583392	0.538349
	10	0.583799	0.538621
	Average	0.557866	0.538566

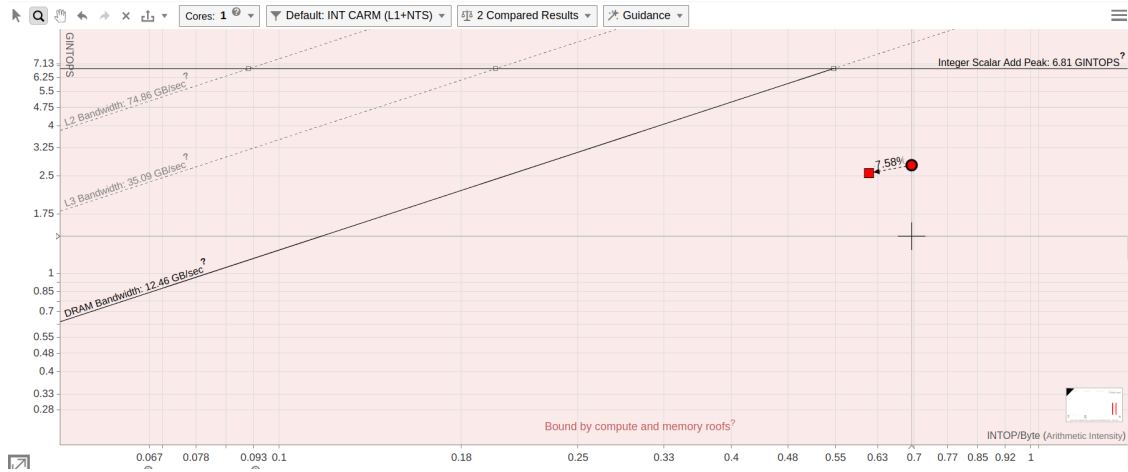


Figure 4: `lsal_plus1` and `lsal_outside` with $N = 32$ and $M = 65536$.

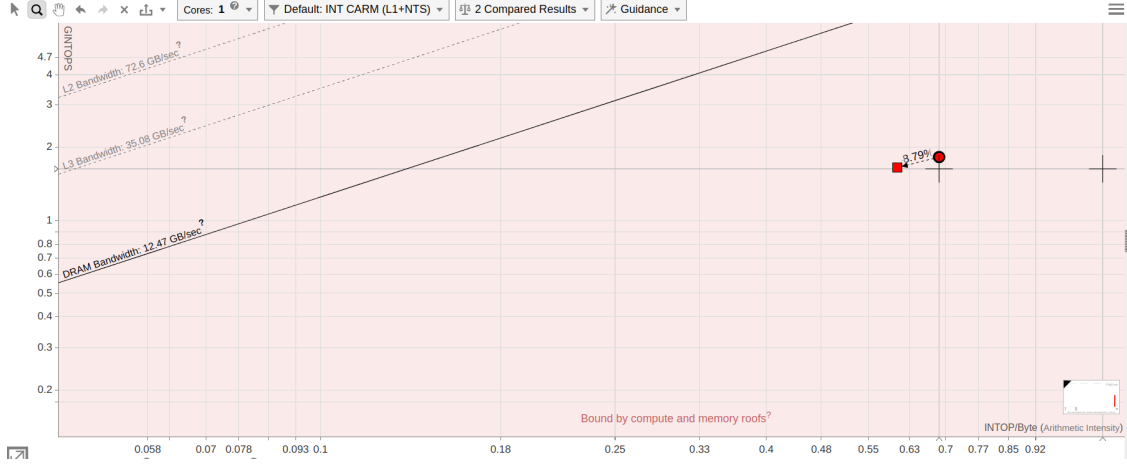


Figure 5: `lsal_plus1` and `lsal_outside` with $N = 256$ and $M = 65536$.

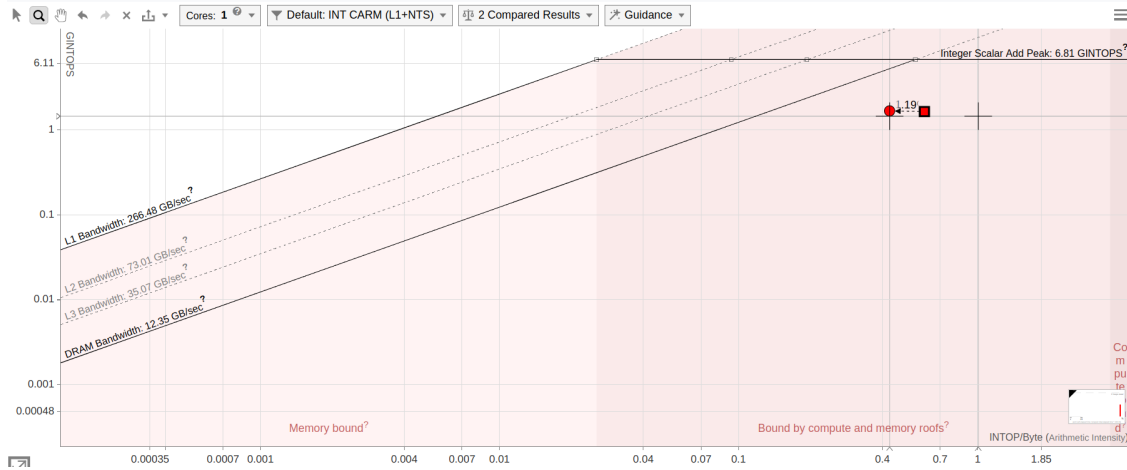


Figure 6: `lsal_plus1` and `lsal_outside` with $N = 256$ and $M = 300000$.

Once again, Intel Advisor data confirms this improvement, showing a further upward and rightward shift on the Roofline chart compared to `lsal_plus1`. This indicates that by moving invariant calculations, such as the row offset and database character lookups, outside the inner loop, redundant operations were reduced and the kernel's throughput improved. By minimizing these inner-loop overheads, the `lsal_outside` variant achieves the highest arithmetic intensity and execution speed, making it the optimal baseline for subsequent architectural optimizations.

2 Arm CPU

2.1 LSAL implementation on the Arm CPU

The optimized x86 implementation (`lsal_outside.cpp`) was cross-compiled and executed on the Zedboard’s Arm Cortex-A9 to establish a performance baseline on a low-power embedded processor. Due to the platform’s limited profiling environment, execution times were measured using the standard `clock()` function. The average execution times across multiple runs are reported below.

Table 3: Average execution time of the optimized LSAL software implementation on the Arm Cortex-A9 processor.

Input Size	Average Execution Time (s)
$N = 32, M = 65536$	0.0727
$N = 256, M = 65536$	0.5609
$N = 256, M = 300000$	2.5280

2.2 Profiling and Performance Analysis on the Arm CPU

Lacking advanced profiling tools like Intel Advisor on the Zedboard, our performance analysis relies on comparing the timing measurements between the embedded Arm processor and the x86 CPU.

Table 4: Comparison between x86 and Arm execution times.

Input Size	x86 Average Time (s)	Arm Average Time (s)	Arm / x86 Ratio
$N = 32, M = 65536$	0.014536	0.0727	$\sim 5.00\times$
$N = 256, M = 65536$	0.117629	0.5609	$\sim 4.77\times$
$N = 256, M = 300000$	0.538566	2.5280	$\sim 4.69\times$

Performance Analysis: The Arm Cortex-A9 is significantly slower, operating at roughly $5\times$ the execution time of the x86 processor. This structural performance gap is driven by the Arm’s lower clock frequency, smaller cache hierarchy, reduced memory bandwidth, and lack of aggressive out-of-order execution capabilities.

Scalability as Inputs Increase: The relative execution time ratio (Arm/x86) drops progressively as the input size increases. The most noticeable drop occurs when moving from the small to the medium input (from $5.00\times$ to $4.77\times$), as the data begins to exceed the x86 processor’s fast L1/L2 caches, forcing it to fetch from slower memory levels. For the largest tested input ($N = 256, M = 300000$), the massive matrix size completely overwhelms the x86’s Last-Level Cache (LLC), causing it to stall for main memory (DRAM) accesses. At this point, both processors become heavily memory-bound, causing the architectural gap to narrow slightly and the ratio to stabilize around $4.69\times$.

3 Hardware

3.1 Step 1: Basic Hardware Porting (lsal_serial)

The transition from the software environment to the FPGA fabric begins by adapting the optimized CPU version, `lsal_outside` (now referred to as `lsal_serial` in the hardware context), into a baseline HLS kernel. At this initial stage, the objective is to establish communication between the ARM processor and the programmable logic, and to perform basic memory access optimizations without altering the fundamental row-by-row execution schedule.

First, the function arguments are mapped to hardware interfaces using `#pragma HLS INTERFACE`. The input sequences and output matrices are mapped to `m_axi` (Master AXI4) ports, allowing the accelerator to read and write directly to the external DDR memory. The control signals and scalar arguments are mapped to `s_axilite` (AXI4-Lite) ports so the ARM CPU can configure and start the kernel execution.

To mitigate the high latency of external memory accesses, we introduced local buffers and pipelining. Instead of fetching characters from the `query` string directly from the DRAM in every iteration, the entire sequence is loaded once into a local array (`local_string1`) using a pipelined loop (`LOAD_S1`). Similarly, the `database` character for the current row is fetched once and stored in `string2_temp`.

Crucially, to compute the Smith–Waterman cell values efficiently without redundant DRAM reads, the dynamic programming dependencies are managed locally. The `north` dependency is fetched from a local buffer (`prev_row`), while the `west` and `northwest` values are tracked using simple registers (`local_west` and `local_northwest`) that update in every cycle. The `#pragma HLS PIPELINE II=1` directive is applied to the inner `COL_LOOP`, instructing the compiler to initiate a new cell computation every clock cycle.

Finally, instead of writing each computed score and direction back to the external matrices one by one, the results are buffered locally in `prev_row` and `local_dir_row`. At the end of each row iteration, the standard C `memcpy` function is used. In HLS, `memcpy` is synthesized into highly efficient, contiguous burst writes over the AXI bus, drastically improving memory bandwidth utilization compared to element-by-element transactions.

```
// Interface definitions for AXI memory and control
// Define that matrices will communicate with main memory (DDR) via AXI Master bus (
m_axi)
#pragma HLS INTERFACE m_axi port=string1 offset=slave bundle=gmem0
#pragma HLS INTERFACE m_axi port=string2 offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=similarity_matrix offset=slave bundle=gmem2
#pragma HLS INTERFACE m_axi port=direction_matrix offset=slave bundle=gmem3
#pragma HLS INTERFACE m_axi port=max_index offset=slave bundle=gmem4
// Define control ports (s_axilite) for the ARM CPU to send commands
#pragma HLS INTERFACE s_axilite port=string1 bundle=control
#pragma HLS INTERFACE s_axilite port=string2 bundle=control
#pragma HLS INTERFACE s_axilite port=similarity_matrix bundle=control
#pragma HLS INTERFACE s_axilite port=direction_matrix bundle=control
```

```

#pragma HLS INTERFACE s_axilite port=max_index bundle=control
#pragma HLS INTERFACE s_axilite port=return bundle=control

// Local query cache
char local_string1[N];
#pragma HLS ARRAY_PARTITION variable=local_string1 complete

LOAD_S1: for (int i = 0; i < N; i++) {
    #pragma HLS PIPELINE II=1
    local_string1[i] = string1[i];
}

// DP dependency tracking and pipelined inner loop
ROW_LOOP: for (j = 1; j <= M; j++) {
    //...
    COL_LOOP: for (i = 1; i <= N; i++) {
        #pragma HLS PIPELINE II=1
        north = prev_row[i];
        west = local_west;
        northwest = local_northwest;

        // ... [Score computation] ...

        local_west = val;
        local_northwest = north;
        prev_row[i] = val;
        local_dir_row[i] = dir;
    }
}

// Burst write to external DRAM
memcpy(&similarity_matrix[row_offset + 1], &prev_row[1], N * sizeof(int));
memcpy(&direction_matrix[row_offset + 1], &local_dir_row[1], N * sizeof(short));

```

Serial Kernel Execution Results

Total Time (ms)	CU Total Time (ms)	Clock Freq (MHz)	LUT	Registers	BRAM
254.111	178.206	142.000	4,089 (7.69%)	6,182 (N/A)	6 (4.29%)

Settings

Summary

Kernels & Compute Units

Kernel Execution

Top Kernel Execution

Compute Unit Utilization

Kernel Data Transfers

Kernel Transfer

Top Kernel Transfer

Host Data Transfers

Host Transfer

Top Memory Writes

Top Memory Reads

API Calls

Kernel Execution

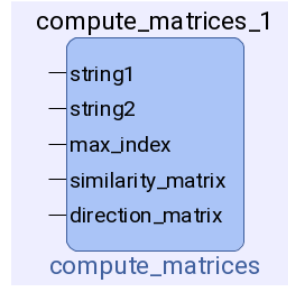
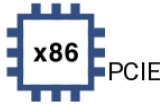
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
 compute_matrices	1	254.111	254.111	254.111	254.111

Top Kernel Execution

Kernel	Kernel Instance Address	Context ID	Command Queue ID	Device	Start Time (ms)	Duration (ms)
 compute_matrices	0x532fb0	0	0	edge-0	205.869	254.111

Compute Unit Utilization

CU Unit	Kernel	Device	Calls	Dataflow Execution	Max Parallel Executions	Dataflow Acceleration	CU Utilization (%)	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	Clock Freq (MHz)
compute_matrices_1	compute_matrices	edge-0	1	Yes	1	1.000000x	70.129	178.206	178.206	178.206	178.206	142.000



Profile

Calls: 1
Utilization: 70.129 %
Total: 178.206 ms
Average: 178.206 ms

Name	Kernel	LUT (% Used)	Register (% Used)	BRAM (% Used)	URAM (% Used)	DSP (% Used)	Calls	CU Utilization (%)	Total Time (ms)	Avg Time (ms)
compute_matrices_1	compute_matrices	4,089 (7.69 %)	6,182 (N/A)	6 (4.29 %)	N/A	0 (0.0 %)	1	70.129 %	178.206	178.206

3.2 Step 2: Anti-Diagonal Scheduling and Datapath Unrolling (lsal_diag)

While `lsal_serial` successfully executes on the FPGA and utilizes burst memory transfers, its performance is fundamentally limited by the software-centric row-by-row execution. In this standard schedule, computing cell (i, j) depends directly on the previously computed cell $(i - 1, j)$, forcing the hardware to process elements sequentially.

To unlock the spatial parallelism of the FPGA, the implementation was restructured into `lsal_diag`, adopting an anti-diagonal execution schedule. Since cells lying on the same anti-diagonal ($k = i + j$) do not share dependencies, they can be calculated entirely concurrently. This transforms the main execution flow: the outer loop (`LOOP_K`) iterates over the diagonals, while the inner loop (`LOOP_I`) evaluates up to $N = 32$ cells simultaneously.

To support this parallel computation, the local storage architecture was completely redesigned. The single `prev_row` buffer was replaced by three dedicated diagonal line buffers: `buf_prev_prev`, `buf_prev`, and `buf_curr`. However, standard FPGA Block RAMs (BRAMs) have only two memory ports, meaning they can only serve two read or write requests per clock cycle. To bypass this limitation, we applied the `#pragma HLS ARRAY PARTITION complete dim=1` directive to these buffers, as well as to `local_string1` and `local_dir_diag`.

Complete partitioning shatters these arrays into individual, independent flip-flop registers. With separate hardware access to all 32 registers, we could safely apply the `#pragma HLS UNROLL` directive to all inner loops, including `LOAD_S1`, `INIT_BUF`, `LOOP_I`, `WRITE_DRAM`, and `DATA_ROTATION`.

Finally, compared to the serial version, three major practical changes were made to the code structure to avoid hardware delays and critical paths:

1. Using `local_string2`: In the serial version, the database character was read directly

during the cell computation. If we did this for 32 cells concurrently, the external memory would stall. To fix this, we created a small, fast local array called `local_string2`. Before the main math loop begins, the `LOAD_DIAG` loop fetches exactly the 32 database characters needed for the current diagonal. This way, when the computation starts, the hardware has instant access to all necessary data.

2. Extracting the Maximum Search (`FIND_MAX_DIAG`): In the serial version, we checked if we found a new maximum score immediately after calculating each cell. However, if 32 cells are calculated at the exact same time, having them all try to update a single "maximum" variable simultaneously creates a massive traffic jam in the hardware logic (a deep critical path). To solve this, we took the maximum search completely out of the main compute loop. The hardware now calculates all 32 cells first, stores them in `buf_curr`, and only then runs the `FIND_MAX_DIAG` loop to find the highest score safely.

3. Flat IFs: Furthermore, a critical datapath optimization was applied to the scoring logic. In `lsal_serial`, the maximum score was determined using a sequence of sequential `if` statements. In hardware, this created a chained critical path (the second comparison waits for the first, the third waits for the second), severely restricting the clock frequency. In `lsal_diag`, this was replaced by a flat `if -- else if` structure. This forces the synthesizer to map the logic into a parallel Priority Encoder (a multiplexer tree) that evaluates all possible paths (NW, N, W) concurrently, preserving the correct software tie-breaking priorities without chaining the logic depth.

```
// Complete partitioning of diagonal buffers and strings into registers
char local_string2[N];
int buf_prev_prev[N + 1];
int buf_prev[N + 1];
int buf_curr[N + 1];
#pragma HLS ARRAY_PARTITION variable=local_string2 complete dim=1
#pragma HLS ARRAY_PARTITION variable=buf_prev_prev complete dim=1
// ...

LOOP_K: for (k = 2; k <= (N + M); k++) {
    #pragma HLS PIPELINE

    // Pre-load current diagonal's database characters
    LOAD_DIAG: for (o = 0; o < N; o++) {
        // ... bounds checking ...
        local_string2[o] = string2[j - 1];
    }

    // Fully unrolled inner loop computes 32 cells concurrently
    LOOP_I: for (i = 1; i <= N; i++) {
        #pragma HLS UNROLL
        // ... Parallel tie-breaking multiplexer logic ...
        if (test_nw >= test_n && test_nw >= test_w && test_nw > 0) {
            val = test_nw; dir = NORTH_WEST;
        } else if (test_n >= test_w && test_n > 0) {
            val = test_n; dir = NORTH;
        } // ...
    }
}
```

```

// Unrolled maximum tracking
FIND_MAX_DIAG: for (i = 1; i <= N; i++) {
    #pragma HLS UNROLL
    if (j >= 1 && j <= M) {
        if (buf_curr[i] > local_max_val) {
            local_max_val = buf_curr[i];
            local_max_index = j * (N + 1) + i;
        }
    }
}
}

```

The Initiation Interval (II) Bottleneck: Despite these aggressive unrolling and logic optimizations, `lsal_diag` fails to achieve the target throughput, with the HLS synthesizer reporting a poor Initiation Interval of `II=32`. This occurs for two major reasons dictated by the code structure: First, inside the unrolled `LOAD_DIAG` and `WRITE_DRAM` loops, the hardware attempts to perform 32 independent reads and 64 independent writes directly to the external memory (`string2`, `similarity_matrix`, `direction_matrix`) in the same cycle. Standard AXI memory ports cannot sustain this without wide coalescing. Second, the `FIND_MAX_DIAG` loop, although unrolled, tries to update a single scalar variable (`local_max_val`) across all 32 lanes. This forces the synthesizer to create a deep, sequential 32-level combinational chain to prevent race conditions. These two critical architectural flaws necessitate the advanced overhauls introduced in the final step.

Anti-Diagonal 1st Version Kernel Execution Results

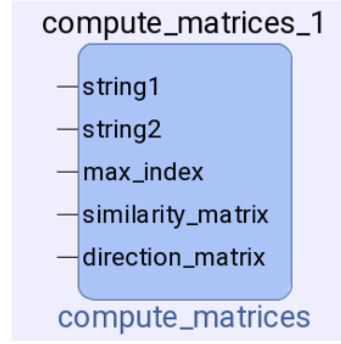
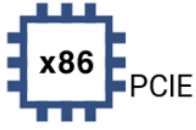
Total Time (ms)	CU Total Time (ms)	Clock Freq (MHz)	LUT	Registers	BRAM
99.573	69.359	142.000	14,572 (27.39%)	15,537 (N/A)	21 (15.0%)

Settings		Kernel Execution									
Summary											
Kernels & Compute Units											
Kernel Execution											
Top Kernel Execution											
Compute Unit Utilization											
Kernel Data Transfers											
Kernel Transfer											
Top Kernel Transfer											
Host Data Transfers											
Host Transfer											
Top Memory Writes											
Top Memory Reads											
API Calls											

Kernel Execution		Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
Kernel						
compute_matrices		1	99.573	99.573	99.573	99.573

Top Kernel Execution		Kernel Instance Address	Context ID	Command Queue ID	Device	Start Time (ms)	Duration (ms)
Kernel							
compute_matrices		0x4764c0	0	0	edge-0	204.476	99.573

Compute Unit Utilization		Kernel	Device	Calls	Dataflow Execution	Max Parallel Executions	Dataflow Acceleration	CU Utilization (%)	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	Clock Freq (MHz)
Compute Unit													
compute_matrices_1		compute_matrices	edge-0	1	Yes	1	1.000000x	69.656	69.359	69.359	69.359	69.359	142.000



Profile

Calls: 1
Utilization: 69.656 %
Total: 69.358 ms
Average: 69.358 ms

Name	Kernel	LUT (% Used)	Register (% Used)	BRAM (% Used)	URAM (% Used)	DSP (% Used)	Calls	CU Utilization (%)	Total Time (ms)	Avg Time (ms)
compute_matrices_1	compute_matrices	14,572 (27.39 %)	15,537 (N/A)	21 (15.0 %)	N/A	0 (0.0 %)	1	69.656 %	69.359	69.359

3.3 Step 3: Advanced Datapath, Reduction Tree, and Memory Bandwidth Optimization

This optimization phase focuses on decreasing the II=32 to II=1. Three major architectural breakthroughs were introduced:

1. Reduced Precision (Arbitrary Types) and AXI Bit-Packing: To reduce the memory bandwidth footprint, standard 8-bit `char` types were replaced with 3-bit arbitrary precision integers (`ap_uint<3>`). The characters (A, C, G, T, and padding P) are pre-encoded in software before being sent to the FPGA. Furthermore, to solve the issue of writing 64 separate values to the external memory per cycle, the accelerator now packs the 32 similarity scores and 32 directions into two massive 512-bit wide words (`ap_uint<512>`). This allows the FPGA to perform exactly one ultra-wide burst write to the DRAM per cycle for each matrix, fully utilizing the AXI bus.

2. Shift-Register Window for the Database (`local_string2_window`): In Step 2, the `LOAD_DIAG` loop attempted to read 32 independent characters from the database in a single clock cycle, heavily stalling the DRAM. To fix this, we changed the logic of `local_string2` into a hardware shift register. Because the anti-diagonal moves through the matrix in a predictable sliding motion, the hardware only needs one new character from the database per diagonal. The accelerator now reads a single character from memory, places it at the front of the window, and shifts the rest. This structural reuse provides the 32 parallel lanes with all the necessary data instantly, dropping the read bandwidth from 32 accesses per cycle down to just 1.

```
// Shift register for the database (string2)
ap_uint<3> local_string2_window[N];
#pragma HLS ARRAY_PARTITION variable=local_string2_window complete dim=1
```

```

// Shifting the local_string2_window locally
SHIFT_S2: for (int s = N - 1; s > 0; s--) {
    #pragma HLS UNROLL
    local_string2_window[s] = local_string2_window[s - 1];
}

// Read only ONE next char from the DRAM per diagonal
ap_uint<3> next_char = 0;
if (k - 2 < M) {
    next_char = string2[k - 2];
}
local_string2_window[0] = next_char;

```

3. Parallel Reduction Tree for Max Search: In Step 2, the unrolled FIND_MAX_DIAG loop attempted to compare 32 calculated cells against a single `local_max_val` variable. In hardware, updating one scalar from 32 different parallel sources creates a massive 32-level sequential logic chain (a severe critical path) to avoid race conditions. To eliminate this, we replaced the loop with a 5-level Parallel Reduction Tree. The 32 scores are first gathered into an array, and invalid out-of-bounds cells are masked with a dummy score of -1. Then, pairwise comparisons are executed simultaneously in stages ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). This logic finds the maximum score and its index in just 5 combinational steps ($\log_2 32 = 5$) instead of 32, completely breaking the critical path without stalling the pipeline.

```

// Gather valid scores; mask out-of-bounds padding cells with -1
GATHER_MAX: for (i = 1; i <= N; i++) {
    #pragma HLS UNROLL
    j = k - i;
    if (j >= 1 && j <= M) {
        tree_val[i-1] = buf_curr[i];
        tree_idx[i-1] = j * (N + 1) + i;
    } else {
        tree_val[i-1] = -1;
        tree_idx[i-1] = 0;
    }
}

// Parallel Reduction Tree (32 -> 16 -> 8 -> 4 -> 2 -> 1)
REDUCTION_32_16: for (int s = 0; s < 16; s++) {
    #pragma HLS UNROLL
    if (tree_val[s + 16] > tree_val[s]) {
        tree_val[s] = tree_val[s + 16];
        tree_idx[s] = tree_idx[s + 16];
    }
}

// ... [Subsequent reductions 16->8, 8->4, 4->2, 2->1] ...

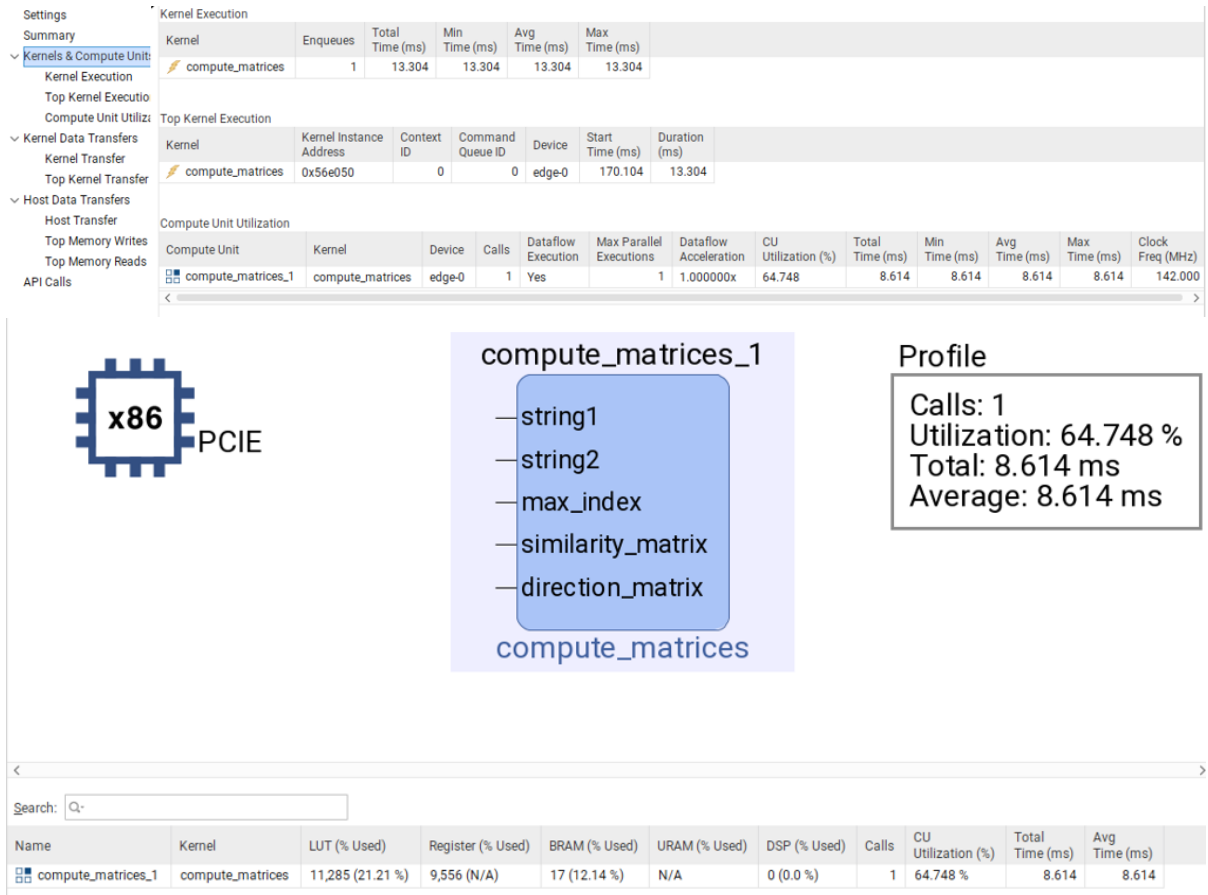
// Update the global scalar safely only once at the end
if (tree_val[0] > local_max_val && tree_val[0] != -1) {
    local_max_val = tree_val[0];
    local_max_index = tree_idx[0];
}

```


Achieving $II=1$ and Future Objectives: With the integration of the arbitrary precision data types, the shift-register window, and the parallel reduction tree, the accelerator successfully achieves the ultimate scheduling target of an Initiation Interval of $II=1$. However, while the pipeline throughput is now structurally optimal (processing one full anti-diagonal per clock cycle), the absolute execution time can still be improved. Our objective for the next optimization phase is to further reduce the overall processing latency and maximize the hardware operating frequency through even deeper architectural refinements.

Anti-Diagonal 2nd Version Kernel Execution Results

Total Time (ms)	CU Total Time (ms)	Clock Freq (MHz)	LUT	Registers	BRAM
13.304	8.614	142.000	11,285 (21.21%)	9,556 (N/A)	17 (12.14%)



3.4 Step 4: Memory Bandwidth Minimization and Output Compression

While Step 3 successfully achieved the target $II=1$, the accelerator's performance and resource utilization were still burdened by excessive external memory transactions. In this step, we focus on strictly minimizing the DRAM write bandwidth by completely eliminating

redundant data transfers and compressing the required outputs. Two major optimizations were implemented:

1. **Elimination of the Similarity Matrix:** In all previous versions, the entire similarity score matrix was written back to the external DRAM. However, from an algorithmic perspective, the traceback phase of the Smith-Waterman algorithm only strictly requires the `direction_matrix` and the starting coordinates (`max_index`). The actual similarity scores are only needed locally to compute the subsequent cells and to continuously track the maximum value. Since our hardware architecture already retains the necessary scores internally within the three sliding diagonal buffers (`buf_prev_prev`, `buf_prev`, `buf_curr`), writing the full score matrix to the external memory is completely redundant. By removing the `similarity_matrix` from the kernel interface and the `WRITE_DRAM` loop, we instantly cut the external memory write traffic.

2. **Direction Matrix Compression (2-bit Encoding):** Previously, the direction of each cell was stored as a 16-bit `short` integer. However, there are only four possible traceback directions (CENTER, NORTH, NORTH-WEST, WEST), which can be perfectly encoded using just 2 bits (`ap_uint<2>`). By changing the `local_dir_diag` array type to 2-bit integers, we dramatically reduced the storage footprint. Consequently, an entire diagonal of 32 directions now requires only 64 bits ($32 \times 2 = 64$). The `direction_matrix` port was resized from an `ap_uint<512>` to an `ap_uint<64>` wide bus, allowing the hardware to pack and write the 32 directions in a single, highly compressed 64-bit burst per cycle.

```
void compute_matrices(
    ap_uint<3> *string1, ap_uint<3> *string2,
    int *max_index,
    ap_uint<64> *direction_matrix) // Similarity matrix removed, width reduced to 64
{
    // ... Interfaces and local buffers ...
    ap_uint<2> local_dir_diag[N + 1]; // Directions encoded in 2 bits

    // ... [Compute and Reduction Tree logic] ...

    // Pack the whole diagonal into a single 64-bit register
    ap_uint<64> packed_direction = 0;

    PACK_LOOP: for (int p = 1; p <= N; p++) {
        #pragma HLS UNROLL
        // Mapping the 2-bit values into the correct bit positions
        packed_direction((p-1)*2 + 1, (p-1)*2) = local_dir_diag[p];
    }

    // Only 1 writing in the DRAM per cycle (Similarity matrix is kept strictly on-chip)
    direction_matrix[k - 2] = packed_direction;

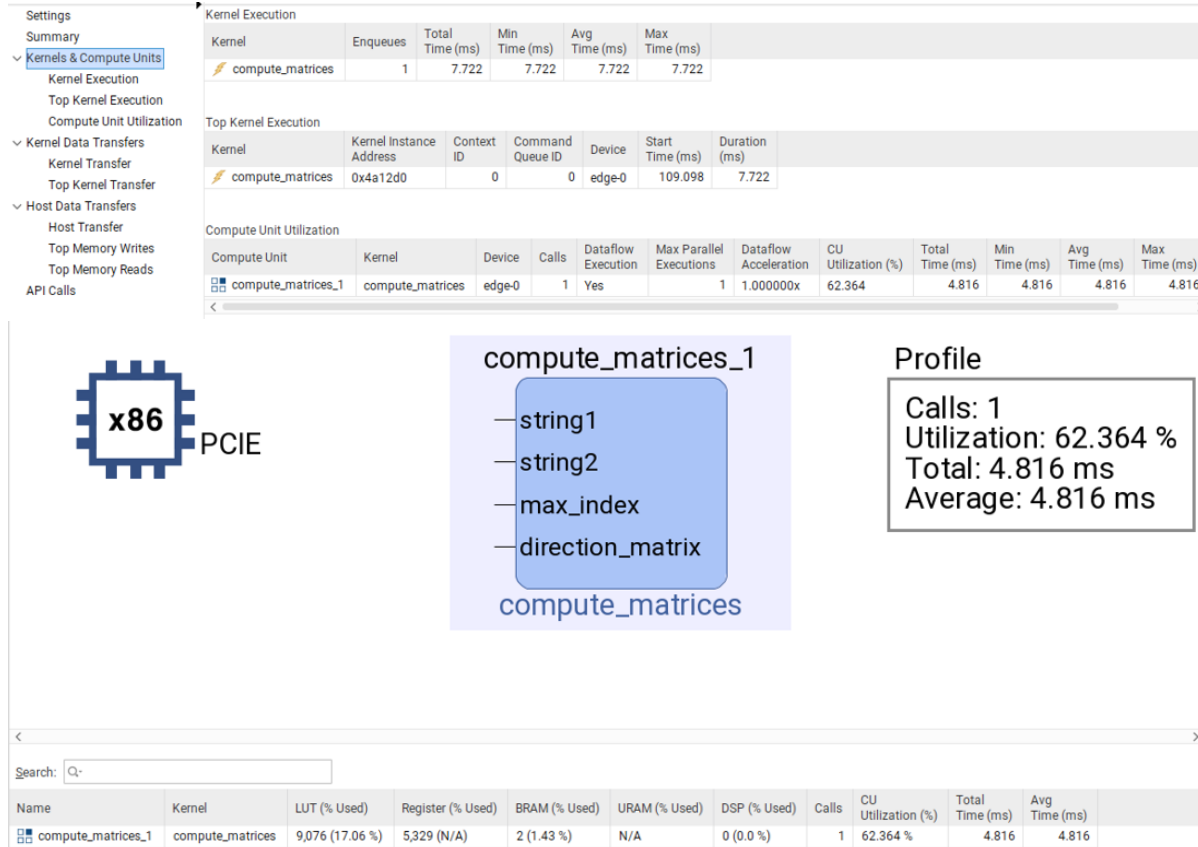
    // ...
}
```

Addressing Remaining Bottlenecks: With the complete removal of the similarity matrix and the 64-bit compression of the direction matrix, the external memory writes have been

drastically minimized. However, while the output bandwidth is now highly optimized, the input read interfaces (`string1` and `string2`) are still reading data inefficiently, leaving room for a final optimization phase to completely maximize the AXI bus potential.

Anti-Diagonal 3rd Version Kernel Execution Results

Total Time (ms)	CU Total Time (ms)	Clock Freq (MHz)	LUT	Registers	BRAM
7.722	4.816	142	9,076 (17.06%)	5,329 (N/A)	2 (1.43%)



3.5 Step 5: Maximizing AXI Bus Potential with Ultra-Wide Input Fetches

While the previous steps successfully minimized the output write bandwidth and achieved a pipeline throughput of $II=1$, the input read interfaces (`string1` and `string2`) were still operating inefficiently. Requesting a single 3-bit character from the external DRAM in every clock cycle severely underutilizes the AXI4 Master bus, which is optimized for large, wide burst transfers. To eliminate this continuous "back-and-forth" traffic to the memory and reduce the absolute execution latency, the input ports were upgraded to `ap_uint<512>`.

By widening the input bus to 512 bits, we unlock the full potential of the memory interface, leading to two massive read optimizations:

1. Single-Cycle Query Fetch (**string1**): Since the query string length is $N = 32$, and each encoded character is 3 bits, the entire query occupies only 96 bits ($32 \times 3 = 96$). By using a 512-bit port, the hardware now fetches the completely encoded query string in a **single memory transaction** (`s1_block = string1[0]`). The 32 individual characters are then instantly unpacked from this local register into `local_string1` using bit-slicing, completely eliminating any further memory reads for the query.

2. Block-Streaming the Database (**string2**): The most critical bottleneck in Step 4 was reading one database character per diagonal directly from the DRAM. Now, a single 512-bit block can hold 170 encoded characters ($512/3 = 170$). Instead of accessing the DRAM every cycle, the hardware fetches a massive 512-bit block and stores it in a local register (`s2_current_block`). The accelerator only requests a new block from the external memory once every 170 cycles (`idx_s2 % 170 == 0`). For the remaining 169 cycles, the required 3-bit character is extracted locally from the register.

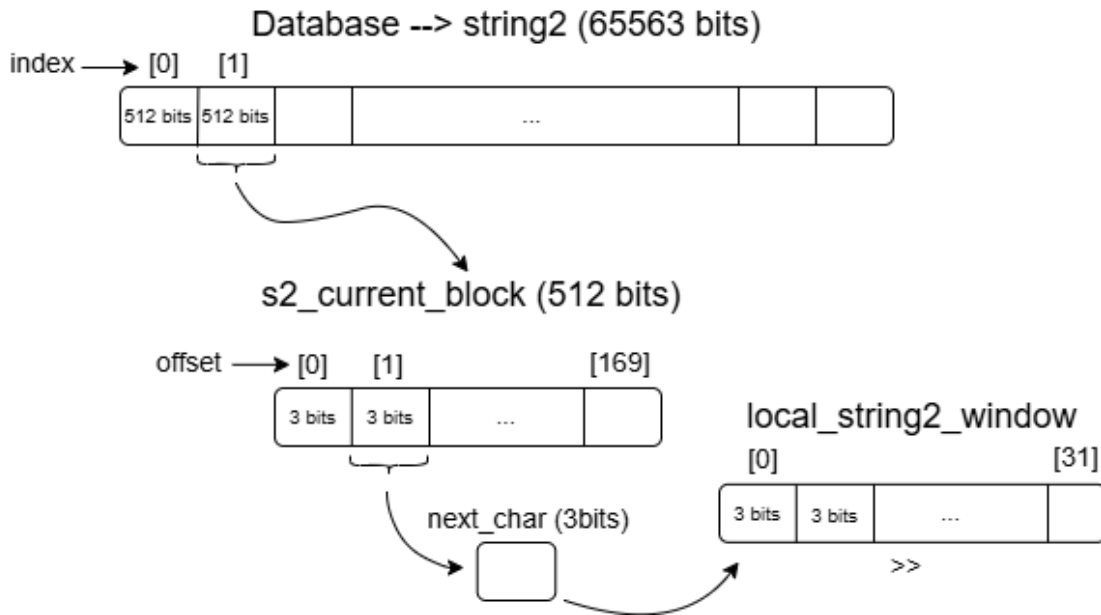


Figure 7: Block-streaming access pattern for the encoded database string using 512-bit input fetches.

```
void compute_matrices(
    ap_uint<512> *string1, // Maximized input bus width
    ap_uint<512> *string2, // Maximized input bus width
    int *max_index,
    ap_uint<64> *direction_matrix)
{
    // ... [Interfaces and partitioned arrays] ...

    // 1. Single-Cycle Query Fetch
    ap_uint<512> s1_block = string1[0];
    LOAD_S1: for (int p = 0; p < N; p++) {
        #pragma HLS UNROLL
        // Unpack 3-bit characters from the 512-bit register
    }
}
```

```

    local_string1[p] = s1_block((p * 3) + 2, p * 3);
}

// ... [Buffer Initializations] ...

ap_uint<512> s2_current_block = 0;

LOOP_K: for (k = 2; k <= (N + M); k++) {
    #pragma HLS PIPELINE

    // ... [Shift Window] ...

    // 2. Block-Streaming the Database (1 Read every 170 cycles)
    int idx_s2 = k - 2;
    ap_uint<3> next_char = 0;

    if (idx_s2 < M) {
        int block_idx = idx_s2 / 170;
        int bit_offset = (idx_s2 % 170) * 3;

        // Access DRAM ONLY when the 512-bit block needs refreshing
        if (idx_s2 % 170 == 0) {
            s2_current_block = string2[block_idx];
        }

        // Instant local unpack
        next_char = s2_current_block(bit_offset + 2, bit_offset);
    }
    local_string2_window[0] = next_char;

    // ... [Core LOOP_I, Direction Packing, and Reduction Tree] ...
}

*max_index = local_max_index;
}

```

Ultra-Wide Input Fetch Kernel Execution Results

Total Time (ms)	CU Total Time (ms)	Clock Freq (MHz)	LUT	Registers	BRAM
1.991	0.729	142.000	10,293 (19.35%)	9,313 (N/A)	11 (7.86%)

Settings

Summary

Kernels & Compute Units

Kernel Execution

Top Kernel Executions

Compute Unit Utilization

Kernel Data Transfers

Kernel Transfer

Top Kernel Transfers

Host Data Transfers

Host Transfer

Top Memory Writes

Top Memory Reads

API Calls

Kernel Execution

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	
compute_matrices	1	1.991	1.991	1.991	1.991	

Kernel Execution

Top Kernel Executions

Compute Unit Utilization

Kernel	Kernel Instance Address	Context ID	Command Queue ID	Device	Start Time (ms)	Duration (ms)
compute_matrices	0x4eba38	0	0	edge-0	105.832	1.991

Kernel Data Transfers

Kernel Transfer

Top Kernel Transfers

Host Data Transfers

Host Transfer

Top Memory Writes

Top Memory Reads

API Calls

Compute Unit Utilization

Compute Unit	Kernel	Device	Calls	Dataflow Execution	Max Parallel Executions	Dataflow Acceleration	CU Utilization (%)	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	Clock Freq (MHz)
compute_matrices_1	compute_matrices	edge-0	1	Yes	1	1.000000x	36.608	0.729	0.729	0.729	0.729	142.000

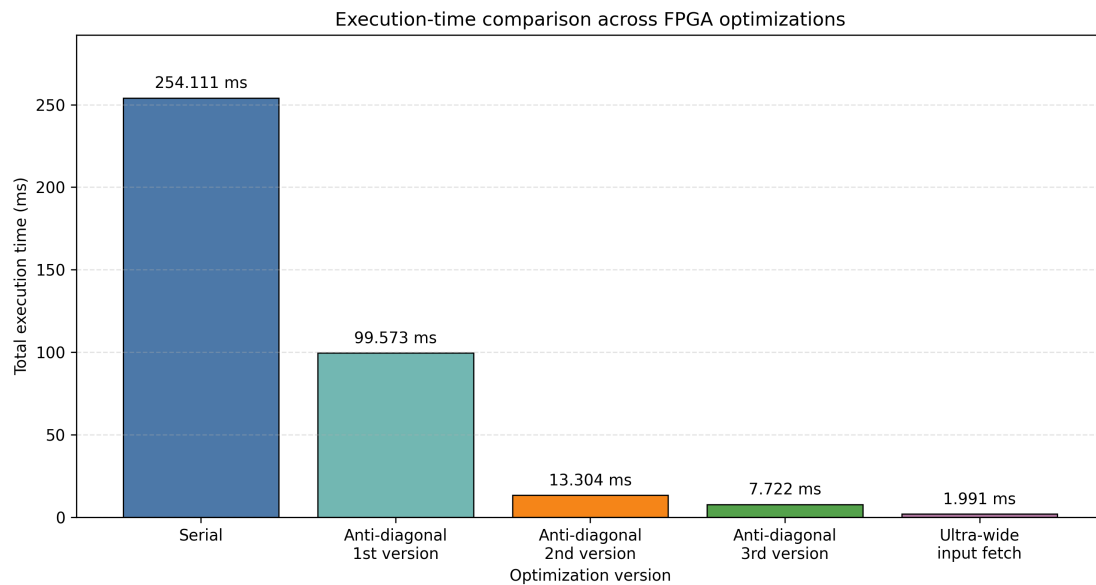
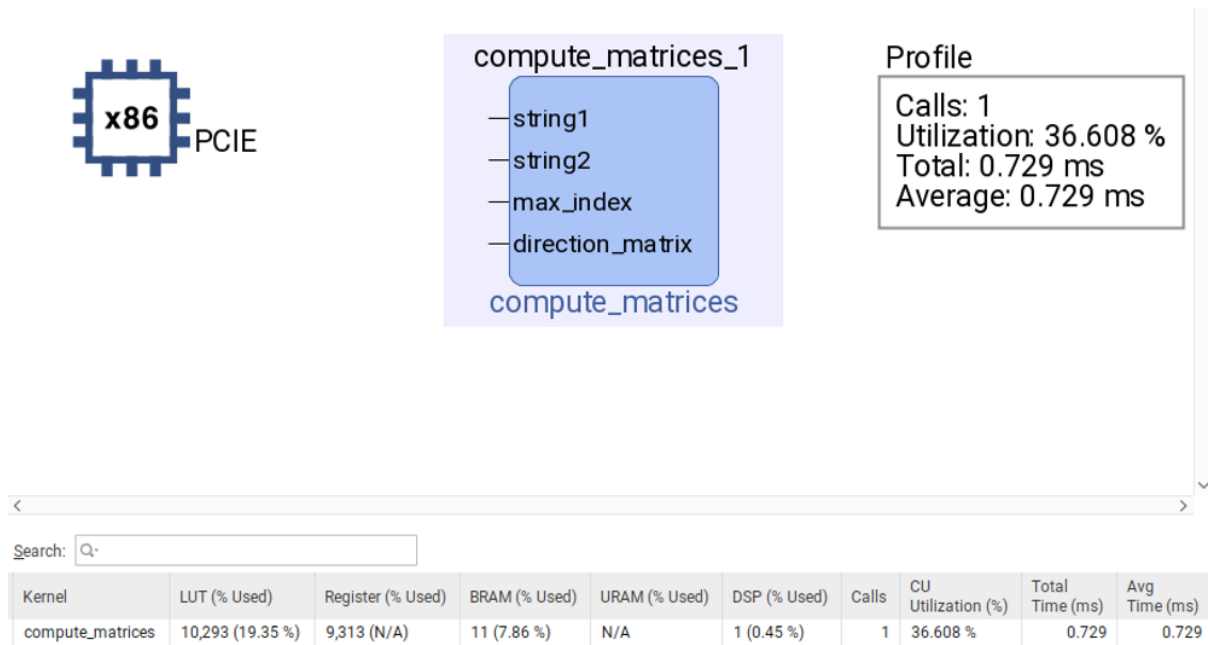


Figure 8: Total execution-time comparison across all FPGA optimization versions.